

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA1001	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	M. Srilakshmi	Reg. No.:	192525404
Date:	01/10/2026	Duration:	60 minutes

Code of Conduct

I M. Srilakshmi (192525404) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

M. Srilakshmi

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
 - Justify why the selected methodology/framework is suitable for the proposed system.
 - Explain how it supports successful project development and maintenance.
- 6. Discuss Expected Outcomes and Limitations**
- Describe the expected benefits of the proposed solution.
 - Explain how the solution addresses the identified challenges.
 - Discuss potential risks, implementation challenges, and limitations.
 - Suggest possible future enhancements.

Predictive Analytics System for Early Detection of Land Acquisition Delays

1. Understanding the Problem Statement

1.1 Explain the Problem in my own words

- Large infrastructure projects (highways, railways, metros, industrial corridors) often stall because the required land is not handed over on time.
- Delays come from many sources: title disputes, compensation disagreements, litigation, missing documents, farmer or community resistance, and slow approvals.
- Today these delays are noticed only after a deadline is missed. By then, cost and time overruns have already occurred.
- The proposed system uses historical and live data to predict which land parcels or projects are likely to be delayed, so action can be taken early.

1.2 Identify the Objectives of the proposed system

- To Predict the probability and expected duration of delay for each parcel, village, or project package.
- To Identify the root causes and risk factors behind each predicted delay.
- To Give early-warning alerts to decision makers.
- To Provide dashboards for monitoring the acquisition status in real time.
- To Recommend mitigation actions based on similar past cases.
- To Improve transparency, accountability, and planning accuracy.

1.3 State the Scope of the problem

- *In scope:* data collection and integration, delay-risk prediction, alerts, dashboards, reports, and recommendations for the acquisition stages (notification, survey, valuation, award, payment, possession).
- *Target users:* government departments and project authorities such as NHAI, railways, and state revenue departments.

- *Out of scope:* making legal decisions, determining compensation amounts, and physical construction management.

2. Identify the Key Stakeholders and Challenges

2.1 Identify all the Stakeholders involved in the system

- **Project authority (NHAI, Railways, Metro, state agencies):** defines land requirements and tracks project timelines and budgets.
- **Land Acquisition Officer / Collector:** issues notifications, conducts surveys, and passes awards.
- **Revenue department / Land records office:** maintains ownership records, maps, and mutation details.
- **Landowners and affected families:** provide consent, file objections, and receive compensation.
- **Courts and legal bodies:** resolve title disputes and compensation litigation.
- **Contractors and project engineers:** depend on land handover to start work.
- **Finance department:** releases and monitors compensation funds.
- **System administrators / data analysts:** maintain the system and models.
- **Government oversight and policymakers:** monitor performance and set policy.

2.2 Explain the Roles and Responsibilities of Each Stakeholder

For the **Predictive Analytics System for Early Detection of Land Acquisition Delays**, the major stakeholders and their responsibilities are:

Stakeholder	Roles and Responsibilities
Government Authorities	Provide land acquisition records, monitor acquisition progress, review predicted delay risks, and take necessary administrative actions.
Project Managers	Monitor project-wise acquisition status, analyze high-risk cases, prioritize delayed cases, and coordinate corrective actions.
Land Acquisition Officers	Update land details, ownership information, documentation, compensation status, legal issues, and approval progress.
Data Analysts	Collect, clean, preprocess, and analyze historical land acquisition data and identify factors contributing to delays.
Machine Learning Engineers	Develop, train, test, and improve the predictive model used to identify potential acquisition delays.
System Administrators	Manage system configuration, user access, database maintenance, security, backups, and system availability.
Authorized Users	Access dashboards, view predictions and alerts, search acquisition records, and generate reports based on their permissions.
Landowners / Affected Parties	Provide required land-related documents and information and participate in the applicable acquisition and compensation processes.
Development Team	Design, develop, test, deploy, maintain, and improve the complete predictive analytics platform.

2.3 Discuss the Major challenges and issues faced by the stakeholders

- Poor, outdated, or inconsistent land records.
- Data scattered across departments, in different formats (paper and digital).
- Disputes over ownership and inheritance.

- Disagreement over compensation rates.
- Court cases and stay orders.
- Delayed fund release.
- Lack of coordination between agencies.
- Resistance from local communities and rehabilitation issues.
- Lack of real-time visibility for senior officials.

3. Analysis of the Current Approach

3.1 Describe Existing process or current process

- Manual or semi-digital tracking using files, spreadsheets, and periodic review meetings.
- Status reports prepared by field officers and sent upward.
- Some portals such as PRAGATI and PARIVESH exist for project monitoring, and states run their own land record systems.

3.2 Identify its Strengths and Limitations

- Established legal and procedural framework.
- Local officers have strong ground knowledge.
- Basic digitization of land records is already in progress.
- Reactive: problems are found after delays occur.
- Dependence on manual reporting, which causes errors and late updates.
- No systematic use of historical data.
- Information stays in silos between departments.
- No risk scoring or prioritization of problem parcels.
- Decisions rely on individual experience rather than data.

3.3 Analyze the problems or Gaps that need to be addressed

- No predictive capability.
- No unified data platform.
- No automated alerts.
- No standardized root-cause analysis.
- Limited transparency and accountability.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Design a software-based solution for the given problem

A web-based, AI-driven platform that integrates acquisition data, applies machine learning models to predict delays, and presents risk insights through dashboards and alerts.

Architecture diagram



4.2 Describe the major functional modules of the proposed system.

- **Data Integration Module:** collects data from land records, GIS, court case systems, finance systems, and field reports.
- **Data Preprocessing Module:** cleaning, validation, deduplication, and feature engineering.
- **Prediction Engine:** ML models (such as Random Forest, XGBoost, and survival analysis) for delay probability and duration.
- **Risk Scoring and Root-Cause Module:** assigns a Low, Medium, or High risk score with key contributing factors.
- **Alert and Notification Module:** sends SMS, email, and in-app alerts when a risk threshold is crossed.
- **Dashboard and GIS Visualization Module:** map-based view of risky parcels, stage-wise progress, and trends.
- **Recommendation Module:** suggests mitigation steps based on similar past cases.
- **Reporting Module:** automated periodic reports.
- **User and Access Management Module:** role-based access and audit logs.

4.3 Identify the Functional and Non-functional requirements

Functional requirements

- Upload and import data from multiple sources.
- Predict delay risk for each parcel and project.
- Display dashboards, maps, and filters.
- Generate alerts and reports.
- Allow officers to update status and add remarks.
- Maintain history and audit trails.

Non-Functional requirements

- **Performance:** dashboards load within a few seconds.
- **Scalability:** supports thousands of parcels and multiple projects.
- **Availability:** high uptime of about 99%.
- **Security:** encryption, authentication, and authorization.
- **Usability:** simple interface, regional language support.
- **Maintainability:** well-documented, modular code.
- **Interoperability:** APIs for integration with government systems.

4.4 Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

Modularity: independent modules (ingestion, ML, alerts, UI) connected through APIs.

- **Scalability:** cloud deployment, microservices, and horizontal scaling.
- **Maintainability:** clean architecture, version control, automated testing, and documentation.
- **Reliability:** data validation, backups, fault tolerance, and model monitoring.
- **Usability:** intuitive dashboards, role-based views, and training support.
- **Security:** role-based access control, data encryption, audit trails, and compliance with data protection laws.

5. Justify the Chosen Methodology/Framework

5.1 Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

Requirements will evolve as stakeholders see early versions and data issues surface.

- Data quality and model accuracy are uncertain at the start, so iterative improvement is needed.
- Frequent feedback from officers ensures the system fits real workflows.
- Working increments can be delivered early (for example, a dashboard in sprint 1–2, prediction in sprint 3–4).

5.2 Justify why the selected methodology/framework is suitable for the proposed system.

- Short sprints (2–4 weeks) with a prioritized product backlog.
- Regular sprint reviews with stakeholders.
- Continuous integration and testing improve quality.
- Easy to add new features and retrain models in later sprints.

- Retrospectives improve the team process.
- Can be combined with DevOps/MLOps for automated deployment and model monitoring.

5.3 How It Supports Successful Project Development and Maintenance

The **Agile methodology** supports the successful development and maintenance of the **Predictive Analytics System for Early Detection of Land Acquisition Delays** in the following ways:

1. **Incremental Development** – The system can be developed in small modules such as data collection, preprocessing, prediction, risk classification, dashboard, and reporting.
2. **Continuous Feedback** – Feedback from project managers, land acquisition officers, and authorized users can be collected after each development iteration.
3. **Early Error Detection** – Regular testing helps identify problems in data processing, prediction accuracy, system functionality, and security at an early stage.
4. **Model Improvement** – The machine learning model can be continuously trained and refined when new land acquisition data becomes available.
5. **Flexible Requirements** – New requirements, prediction factors, reports, or dashboard features can be added during later development iterations.
6. **Continuous Maintenance** – Regular updates can improve system performance, security, prediction accuracy, and usability after deployment.
7. **Risk Reduction** – Developing and testing the system in smaller iterations reduces the risk of major failures during final deployment.
8. **Scalability and Reliability** – Continuous improvement and testing help the system handle increasing numbers of land acquisition records and users effectively.

6. Discuss Expected Outcomes and Limitations

6.1 Describe the expected benefits of the proposed solution.

- Early identification of high-risk parcels, ahead of deadlines.
- Reduced project delays and cost overruns.
- Better resource and manpower allocation.
- Data-driven, transparent decision making.
- Improved coordination between departments.
- Faster dispute resolution through targeted intervention.

6.2 Explain how the solution addresses the identified challenges.

- Predictive alerts replace reactive monitoring.
- A unified platform removes data silos.
- Root-cause analysis highlights the real bottlenecks.
- Dashboards give officials real-time visibility.

6.3 Discuss potential risks, implementation challenges, and limitations.

- Poor or incomplete historical data can reduce accuracy.
- Resistance to change from officials and staff.
- Data privacy and security concerns.
- Difficulty integrating with legacy systems.
- Model bias, or drift as policies and conditions change.
- Predictions are probabilistic, not guaranteed.
- Limited digital infrastructure in some regions.
- Cost and need for skilled personnel.

6.4 Suggest possible future enhancements.

Satellite imagery and drone data for encroachment and land-use detection.

- NLP analysis of court orders and grievance documents.
- Mobile app for field officers with offline support.
- Citizen portal for grievance tracking and compensation status.
- Blockchain-based land record verification.
- Chatbot assistant for officials.
- Simulation of "what-if" scenarios for planning.
- Multilingual voice support.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.

Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25